



Реклама. arId: 2VtzqWLLmbd. АО Селектел

Облачный сервер Selectel

с криперами и порталом в Незер

[Исследовать](#)

★ 4.45

Оценка

2086.05

Рейтинг

Selectel

IT-инфраструктура для бизнеса

[Подписаться](#)

Seleditor

3 июл в 08:40

Telegram-бот, который умеет слушать: разработка на grammY



Простой



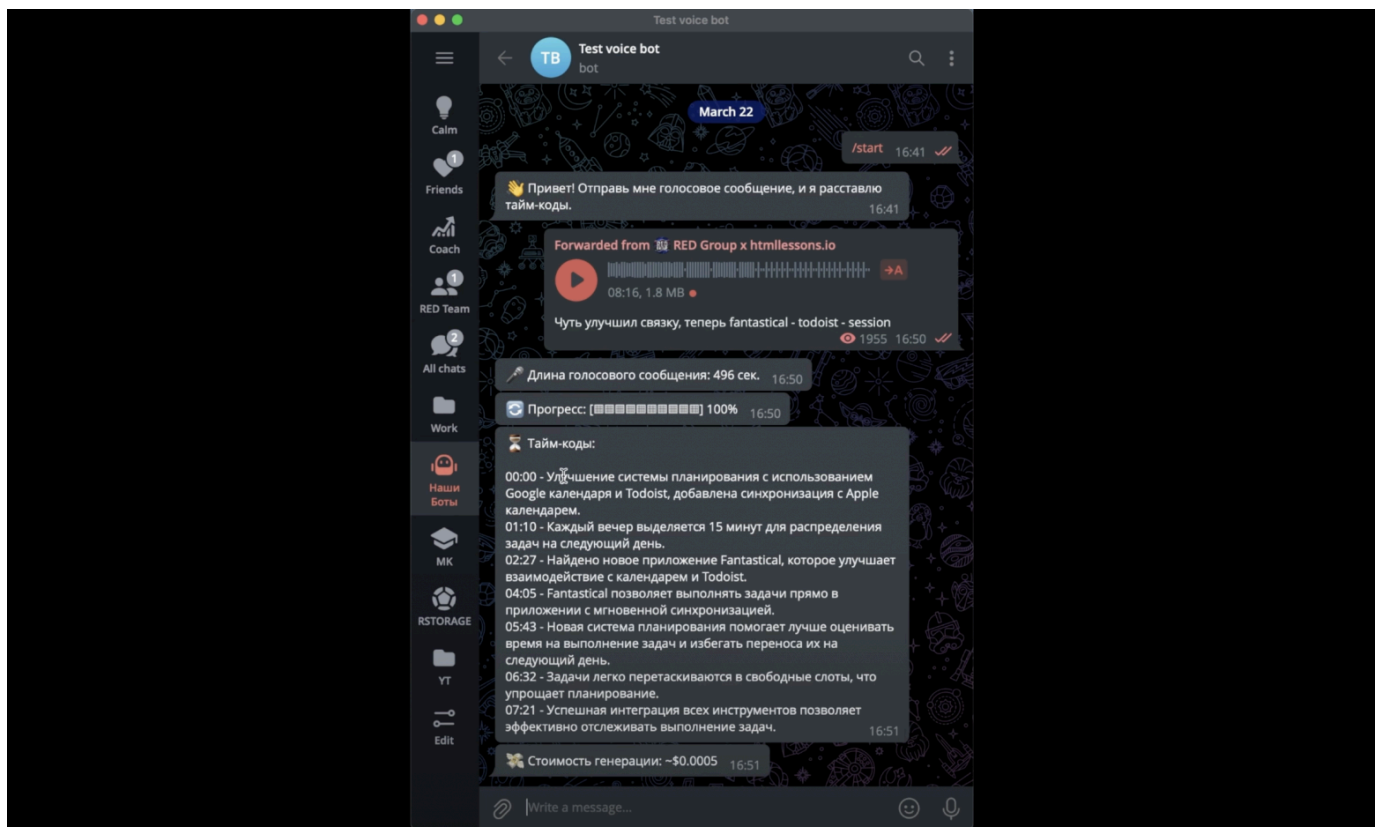
12 мин



5.2K

Блог компании Selectel, JavaScript*, NestJS*, [Мессенджеры*](#), Программирование*

[Тutorial](#)



Представьте: собеседник отправляет голосовое сообщение на пять минут, а вы не можете отвлечься и прослушать все от начала до конца? Что делать? Максим, ведущий канала RED Group, подошел к вопросу творчески и показал, как на базе grammY и SpeechService в NestJS разработать бота, который будет слушать и структурировать по таймкодам голосовые сообщения.

Инструкция будет полезна новичкам, которые только погружаются в работу с Telegram Bot API с помощью JavaScript. Кроме того, в конце материала мы разберем, как задеплоить готового бота на сервер, чтобы он работал вне зависимости от локальной машины. Подробности под катом!

Инициализация проекта

Для начала переходим в терминал и пишем команду:

```
nest new test_voice_bot
```

Далее выбираете пакетный менеджер или оставляете тот, что по умолчанию.

```
> nest new test-voice-red-group
⚡ We will scaffold your app in a few seconds..

? Which package manager would you ❤️ to use? npm
CREATE test-voice-red-group/.eslintrc.js (663 bytes)
CREATE test-voice-red-group/.prettierrc (51 bytes)
CREATE test-voice-red-group/README.md (3340 bytes)
CREATE test-voice-red-group/nest-cli.json (171 bytes)
CREATE test-voice-red-group/package.json (1960 bytes)
CREATE test-voice-red-group/tsconfig.build.json (97 bytes)
CREATE test-voice-red-group/tsconfig.json (546 bytes)
CREATE test-voice-red-group/src/app.controller.ts (274 bytes)
CREATE test-voice-red-group/src/app.module.ts (249 bytes)
CREATE test-voice-red-group/src/app.service.ts (142 bytes)
CREATE test-voice-red-group/src/main.ts (208 bytes)
CREATE test-voice-red-group/src/app.controller.spec.ts (617 bytes)
CREATE test-voice-red-group/test/jest-e2e.json (183 bytes)
CREATE test-voice-red-group/test/app.e2e-spec.ts (630 bytes)

***** Installation in progress ... 📦
```

После создания проекта откройте его в удобном редакторе кода — например, в VSCode.

Реклама. erid: 2Vtzqy3o1sR. АО Селектех

Облачная инфраструктура для ваших проектов



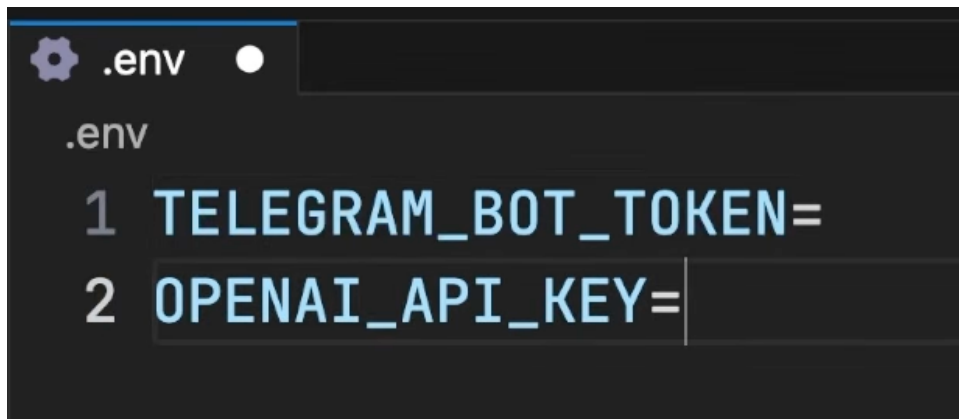
Платите только за реальное потребление!
(pay-as-you-go)

ПОДРОБНЕЕ

Разработка бота

Регистрация API

1. Открываем Telegram и в поиске вбиваем **@BotFather**. Отправляем ему команду `/newbot`.
2. Вписываем уникальный username и получаем ключ — токен для управления ботом. Далее вы можете кастомизировать его профиль: установить имя, аватарку и описание. Как это делать — можно понять, если ознакомитесь с доступным в чате списком команд.
3. Далее необходимо получить доступ к сервисам OpenAI. Для этого переходим на официальную страницу и регистрируемся. Процесс подробно описан в документации.
4. Создаем в корне проекта файл `.env` с переменными окружения, куда вписываем токен от Telegram-бота и ключ от API-сервисов OpenAI:



5. Открываем терминал и устанавливаем необходимые зависимости:

```
bun add @grammyjs/nestjs grammy axios @nestjs/config
```

```
bun add -D @types/express
```

Первая команда устанавливает основные библиотеки для работы с Telegram-ботами и конфигурацией. Среди них:

- `@grammyjs/nestjs` — интеграция библиотеки `Grammy` (для создания Telegram-ботов) с фреймворком `NestJS`;
- `grammy` — основная библиотека для работы с `Telegram Bot API`;
- `axios` — популярная библиотека для выполнения `HTTP`-запросов;
- `@nestjs/config` — модуль для управления конфигурациями в приложениях на `NestJS`.

Вторая — добавляет типы для `Express.js` как зависимость для разработки. Это нужно для поддержки `TypeScript` и автодополнения при работе с `Express`.

Создание базовых вспомогательных файлов

`src/constants.ts`

В данном файле будут размещены пути до внешних API.

```
export const TELEGRAM_API = 'https://api.telegram.org'  
export const OPENAI_API = 'https://api.openai.com/v1'
```

src/prompts/timestamp.prompts.ts

Далее нам необходимо подготовить шаблон промтов, которые будет использовать Telegram-бот для расшифровки поступающих голосовых сообщений через сервисы OpenAI.

```
/**
 * System-промт: описывает поведение и правила для модели
 */
export const TIMESTAMP_SYSTEM_PROMPT = `
Ты – ассистент, который составляет тайм-коды к голосовым сообщениям.
У тебя есть расшифровка текста, разбитая на временные блоки.
Твоя задача – выбрать из каждого блока ОДНУ ключевую идею (если она есть)
и указать её с точным тайм-кодом начала блока.

Правила:
- Не выдумывай тем, которых не было в тексте.
- Не объединяй идеи из разных блоков.
- Не используй больше 10 пунктов.
- Не добавляй "Заключение", "Финал", если этого не было в речи.
- Сохраняй реальный тайминг – не позже времени блока.
- Пропускай блок, если в нём нет ничего важного.

Формат:
00:00 - Введение
00:35 - Почему важно планировать день
01:10 - Проблема прокрастинации
`

/**
 * Генерирует пользовательский промт на основе подготовленного текста
 */
export const buildTimestampUserPrompt = (preparedText: string): string => `
Вот текст, расшифрованный из голосового сообщения. Каждый блок соответствует примерно 30 секундам.
Для каждого блока выдели ключевую идею (если она есть), строго по времени начала блока.

Текст:
${preparedText}
`
```

В файлах далее при вставке могут возникать ошибки, так как мы еще не создали самые

ключевые файлы и они их просто не найдут.

src/telegram/telegram.module.ts

Здесь мы подключаем библиотеки для работы с Telegram и для доступа к .env.

```
import { NestjsGrammyModule } from '@grammyjs/nestjs'
import { Module } from '@nestjs/common'
import { ConfigModule, ConfigService } from '@nestjs/config'
import { AIService } from 'src/services/ai.service'
import { SpeechService } from 'src/services/speech.service'
import { TelegramUpdate } from './telegram.update'

@Module({
  imports: [
    ConfigModule,
    NestjsGrammyModule.forRootAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: async (configService: ConfigService) => ({
        token: configService.get<string>('TELEGRAM_BOT_TOKEN')
      })
    })
  ],
  providers: [TelegramUpdate, SpeechService, AIService]
})
export class TelegramModule {}
```

src/app.module.ts

Теперь отредактируем текущий корневой файл. В нем мы просто подключаем дочерние модули. Среди них есть и те, что мы ранее создали.

```
import { Module } from '@nestjs/common'
import { ConfigModule } from '@nestjs/config'
import { TelegramModule } from './telegram/telegram.module'

@Module({
  imports: [ConfigModule.forRoot(), TelegramModule]
})
export class AppModule {}
```


Реализация распознавания аудио и речи

Создадим новый файл `src/services/speech.service.ts` — он будет содержать `SpeechService` в NestJS. Если коротко, то он делает всю сложную работу за нас и умеет расшифровывать голосовые сообщения из Telegram с помощью OpenAI Whisper. Для этого сервис:

1. загружает голосовой файл с Telegram по переданному пути;
2. создает `FormData` с этим файлом для API OpenAI;
3. отправляет запрос на `whisper-1` модель OpenAI для транскрибации;
4. возвращает полученный текст.

Все ключи (`TELEGRAM_BOT_TOKEN`, `OPENAI_API_KEY`) берутся из переменных окружения через `ConfigService` (файла `.env`).

```
import { Injectable } from '@nestjs/common'
import { ConfigService } from '@nestjs/config'
import axios from 'axios'
import * as FormData from 'form-data'
import { OPENAI_API, TELEGRAM_API } from '../constants'

@Injectable()
export class SpeechService {
  private readonly botToken: string
  private readonly openaiApiKey: string

  constructor(private readonly configService: ConfigService) {
    this.botToken = this.configService.get<string>('TELEGRAM_BOT_TOKEN')
    this.openaiApiKey = this.configService.get<string>('OPENAI_API_KEY')
  }

  async transcribeVoice(filePath: string): Promise<string> {
    const fileUrl = `${TELEGRAM_API}/file/bot${this.botToken}/${filePath}`
    const fileResponse = await axios.get(fileUrl, { responseType: 'stream' })

    const formData = new FormData()
    formData.append('file', fileResponse.data, { filename: 'audio.ogg' })
    formData.append('model', 'whisper-1')

    const response = await axios.post<{ text: string }>(
      `${OPENAI_API}/audio/transcriptions`,
      formData,
      {

```

```

        headers: {
            Authorization: `Bearer ${this.openaiApiKey}`,
            ...formData.getHeaders()
        }
    }
)

return response.data.text
}
}

```

Генерация таймкодов

Далее создадим новый файл `src/services/ai.service.ts` — он будет содержать сервис `AIService` из NestJS, который с помощью OpenAI GPT-4o генерирует улучшенные таймкоды по тексту и длительности аудио. И снова: все уже сделано за нас!

Если коротко, сервис:

1. делит текст на десять равных блоков по словам и времени;
2. формирует вид [MM:SS] текст для каждого блока;
3. отправляет эти блоки в GPT с системным промптом;
4. получает улучшенные таймкоды от модели;
5. считает примерную стоимость запроса (по токенам);
6. возвращает результат и стоимость.

```

import { Injectable } from '@nestjs/common'
import { ConfigService } from '@nestjs/config'
import axios from 'axios'
import { OPENAI_API } from '../constants'
import {
    TIMESTAMP_SYSTEM_PROMPT,
    buildTimestampUserPrompt
} from '../prompts/timestamp.prompts'

interface IOpenAIResponse {
    choices: {
        message: {
            content: string
        }
    }
}

```



```

    }[]
    usage: {
        prompt_tokens: number
        completion_tokens: number
    }
}

@Injectable()
export class AIService {
    private readonly openaiApiKey: string

    constructor(private readonly configService: ConfigService) {
        this.openaiApiKey = this.configService.get<string>('OPENAI_API_KEY')
    }

    async generateTimestamps(
        text: string,
        audioDurationSec: number
    ): Promise<{ timestamps: string; cost: string }> {
        const maxSegments = 10 // Максимум логических блоков для разметки

        // Разбиваем весь текст на отдельные слова
        const words = text.split(/\s+/)

        // Сколько слов и секунд должно быть в каждом блоке
        // Округление вверх — чтобы не потерять слова: 9.6 → 10
        const wordsPerSegment = Math.ceil(words.length / maxSegments)
        // Округление вниз — чтобы не «вывалиться» за длину аудио: 96.5 → 96
        const secondsPerSegment = Math.floor(audioDurationSec / maxSegments)

        // Собираем массив временных блоков текста
        const segments: { time: string; content: string }[] = []

        for (let i = 0; i < maxSegments; i++) {
            // Вычисляем метку времени начала блока
            const fromSec = i * secondsPerSegment
            // padStart - Форматирует числа вида 5 → 05
            // Чтобы мы всегда получали формат MM:SS, а не M:S.
            const fromMin = String(Math.floor(fromSec / 60)).padStart(2, '0')
            const fromSecRest = String(fromSec % 60).padStart(2, '0')
            const time = `${fromMin}:${fromSecRest}`

            // Вырезаем часть текста, относящуюся к текущему блоку
            const start = i * wordsPerSegment
            const end = start + wordsPerSegment
            const content = words.slice(start, end).join(' ')

```

```

    // Добавляем только непустые блоки
    if (content.trim()) {
        segments.push({ time, content })
    }
}

// Объединяем блоки в формат вида: [00:00] текст
const preparedText = segments
    .map(s => `[${s.time}] ${s.content}`)
    .join('\n\n')

// Готовим сообщения для GPT: правила и ввод
const systemMessage = TIMESTAMP_SYSTEM_PROMPT
const userMessage = buildTimestampUserPrompt(preparedText)

// Отправляем запрос в OpenAI Chat API
const response = await axios.post<IOpenAIResponse>(
    `${OPENAI_API}/chat/completions`,
    {
        model: 'gpt-4o-mini',
        messages: [
            { role: 'system', content: systemMessage }, // задаёт поведение
            { role: 'user', content: userMessage } // передаёт текст с блоками
        ],
        temperature: 0.3, // Насколько "свободно" думает модель (0 – строго, 1
        max_tokens: 300 // Ограничиваем объем ответа, чтобы не получить слишком
    },
    {
        headers: {
            Authorization: `Bearer ${this.openaiApiKey}`
        }
    }
)

// Извлекаем ответ GPT
const result = response.data.choices[0].message.content

// Статистика использования токенов (нужна для расчёта стоимости)
const usage = response.data.usage

// Подсчет примерной стоимости запроса
const inputCost = (usage.prompt_tokens / 1_000_000) * 0.15
const outputCost = (usage.completion_tokens / 1_000_000) * 0.6
const total = inputCost + outputCost

```

```

const costText = `💰 Стоимость генерации: ~\${total.toFixed(4)}`

return {
  timestamps: result, // Сами тайм-коды
  cost: costText // Стоимость генерации
}
}
}

```

Разработка основных функций

Заведём новый файл `src/telegram/telegram.update.ts` — в нём будет реализован Telegram-обработчик `TelegramUpdate` на базе `@grammyjs/nestjs`. Он ожидает и принимает голосовые сообщения. После — отвечает с расшифровкой и таймкодами. А именно:

1. Обрабатывает команду `/start` и голосовые сообщения от пользователя в Telegram.
2. Показывает красивый прогресс обработки (обновляется каждые 2–3 секунды).
3. Получает аудио из Telegram и расшифровывает его через `SpeechService`.
4. Отправляет текст в `AIService` для генерации тайм-кодов.
5. Возвращает результат и стоимость генерации пользователю.

```

import { Ctx, InjectBot, On, Start, Update } from '@grammyjs/nestjs'
import { Injectable } from '@nestjs/common'
import { ConfigService } from '@nestjs/config'
import { Api, Bot, Context } from 'grammy'
import { AIService } from '../services/ai.service'
import { SpeechService } from '../services/speech.service'

@Update() // Этот декоратор указывает, что класс слушает события от Telegram
@Injectable()
export class TelegramUpdate {
  private readonly botToken: string

  constructor(
    @InjectBot() private readonly bot: Bot<Context>, // Внедрение Telegram-бота
    private readonly speechService: SpeechService, // Сервис для расшифровки речи
    private readonly aiService: AIService, // Сервис для генерации тайм-кодов
    private readonly configService: ConfigService
  ) {

```

```

    this.botToken = this.configService.get<string>('TELEGRAM_BOT_TOKEN')
  }

  @Start() // Обрабатывает команду /start
  async onStart(@Ctx() ctx: Context): Promise<void> {
    await ctx.reply(
      '👋 Привет! Отправь мне голосовое сообщение, и я расставлю тайм-коды.'
    )
  }

  @On('message:voice') // Обработка голосовых сообщений
  async onVoiceMessage(@Ctx() ctx: Context): Promise<void> {
    let progressMessageId: number | undefined
    let interval: NodeJS.Timeout | undefined
    let percent = 10 // Начальный процент прогресса

    try {
      const voice = ctx.msg.voice
      const duration = voice.duration

      // Получаем путь к файлу голосового сообщения
      const file = await ctx.getFile()

      // Показываем длительность голосового
      await ctx.reply(`🎤 Длина голосового сообщения: ${duration} сек.`)

      // Отправляем первое сообщение с прогрессом
      const progressMsg = await ctx.reply(this.renderProgress(percent))
      progressMessageId = progressMsg.message_id

      // 🕒 Эмулируем "оживший" прогресс — обновляем каждые 2 секунды
      interval = setInterval(
        async () => {
          if (percent < 90) {
            percent += 5
            await this.updateProgress(
              ctx.api,
              ctx.chat.id,
              progressMessageId,
              percent
            )
          }
        },
        duration > 300 ? 3000 : 2000
      )
    }
  }

```

```

// Расшифровываем речь с помощью Whisper
const transcription = await this.speechService.transcribeVoice(
  file.file_path
)

// Отправляем текст в OpenAI и получаем тайм-коды + стоимость
const { timestamps, cost } = await this.aiService.generateTimestamps(
  transcription,
  duration
)

// Останавливаем обновление прогресса
clearInterval(interval)
await this.updateProgress(ctx.api, ctx.chat.id, progressMessageId, 100)

// Отправляем результат
await ctx.reply(
  `⌚ Тайм-коды:\n\n${timestamps}\n\n<i>🗣️ Таймы генерирует нейросеть, `
  {
    parse_mode: 'HTML'
  }
)
await ctx.reply(cost)
} catch (error) {
  clearInterval(interval) // Останавливаем прогресс даже при ошибке
  console.error('Ошибка при обработке голосового:', error.message)
  await ctx.reply('⚠️ Ошибка при обработке голосового сообщения.')
}
}

// Обновление прогресса (редактирует предыдущее сообщение)
private async updateProgress(
  api: Api,
  chatId: number,
  messageId: number,
  percent: number
) {
  await api.editMessageText(chatId, messageId, this.renderProgress(percent))
}

// Отрисовка прогресс-бара с заданным процентом
private renderProgress(percent: number): string {
  const totalBlocks = 10 // Всего 10 ячеек в прогресс-баре
  const blockChar = '█' // Символ, обозначающий "заполненную" ячейку прогресса

  // Вычисляем количество заполненных блоков на шкале

```

```

const filledBlocks = Math.max(1, Math.round((percent / 100) * totalBlocks))

/**
 * 👉 Math.round(...) — округляет до ближайшего целого (например, 3.6 → 4)
 * 👉 (percent / 100) * totalBlocks — переводим процент в количество блоков
 * 👉 Math.max(1, ...) — гарантируем, что хотя бы 1 блок всегда будет показан
 */

const emptyBlocks = totalBlocks - filledBlocks // Остальные блоки считаем как

/**
 * 👉 String.prototype.repeat(n) — повторяет символ n раз
 * Пример: '█'.repeat(4) = '████'
 * Таким образом формируем заполненную и пустую часть визуального прогресса
 */

// Собираем строку вида: 🔄 Прогресс: [████░░░░] 40%
return `🔄 Прогресс: [${blockChar.repeat(filledBlocks)}${'░'.repeat(emptyBlocks)}] ${percent}%`
}
}

```

Отлично! Бота уже можно протестировать, запустив его одной из следующих команд:

```
bun start:dev
```

```
npm run start:dev
```

Чтобы бот работал круглосуточно и без перебоев, его необходимо развернуть на сервере. Один из удобных вариантов — облачная платформа [Selectel](#), которая предоставляет виртуальные серверы в понятной панели управления.

Загрузка кода в репозиторий

Код готов, его можно перенести на облачный сервер с помощью утилиты `scp`, но более удобный вариант — загрузить через Git-репозиторий. Вы можете выбрать любую платформу, принципы везде одни и те же. Но мы коротко разберем процесс на примере самого популярного варианта — GitHub. Подробные инструкции — [в отдельной статье](#).

1. Зарегистрируйтесь на платформе GitHub.

2. Нажмите на кнопку **New**, чтобы создать новый репозиторий.
3. Назовите репозиторий удобным образом. Рекомендуем поставить галочку напротив **Add on README file**, чтобы репозиторий не был пустым и вы сразу получили доступ к его файловой системе. Нажмите кнопку **Create repository**.
4. Мышкой перетащите файлы своего бота в область репозитория — и все загрузится в облако GitHub. Обратите внимание, что `.env` переносить нельзя — в нем записаны ключи для работы с API вашего бота.

Подготовка сервера

Создание сервера в панели управления

1. Первым шагом нужно зарегистрироваться в панели управления Selectel.
2. Далее в верхнем меню переходим во вкладку **Продукты** → **Облачные серверы**. Нажимаем на кнопку **Создать сервер**.

Регион

Новосибирск

Пул

ru-8a

Источник

New — Приложения

Ubuntu 24.04 LTS 64-bit GPU driver

2 ГБ RAM, 10 ГБ Диск

Конфигурация

Фиксированная

Произвольная

Standard

CPU

Memory

GPU

Shared

HighFreq

Конфигурации с процессором 2,2—2,4 ГГц.

☐

Локальный SSD NVMe диск

Загрузочный диск без сетевых задержек. Чтение 12 800 IOPS / Запись 6 400 IOPS

Для облачных серверов с локальным диском недоступна функция заморозки.

1 vCPU

1 ГБ RAM

1 vCPU

2 ГБ RAM

2 vCPU

4 ГБ RAM

2 vCPU

8 ГБ RAM

4 vCPU

16 ГБ RAM

6 vCPU

32 ГБ RAM

8 vCPU

32 ГБ RAM

12 vCPU

48 ГБ RAM

16 vCPU

64 ГБ RAM

Диски

Тип диска

SSD Универсальный

Размер

— 10 +

ГБ

ТБ

Добавить

Цена

Час

День

Месяц

Конфигурация

1 vCPU

21,54 Р

2 ГБ RAM

15,67 Р

10 ГБ Универсальный SSD диск

6,1 Р

Цена

43,31 Р

Сеть

1 Публичные IP

5,88 Р

Итого

49,18 Р

Создать сервер

Отменить

Нужна помощь с созданием сервера?

Ознакомьтесь с [инструкцией](#).

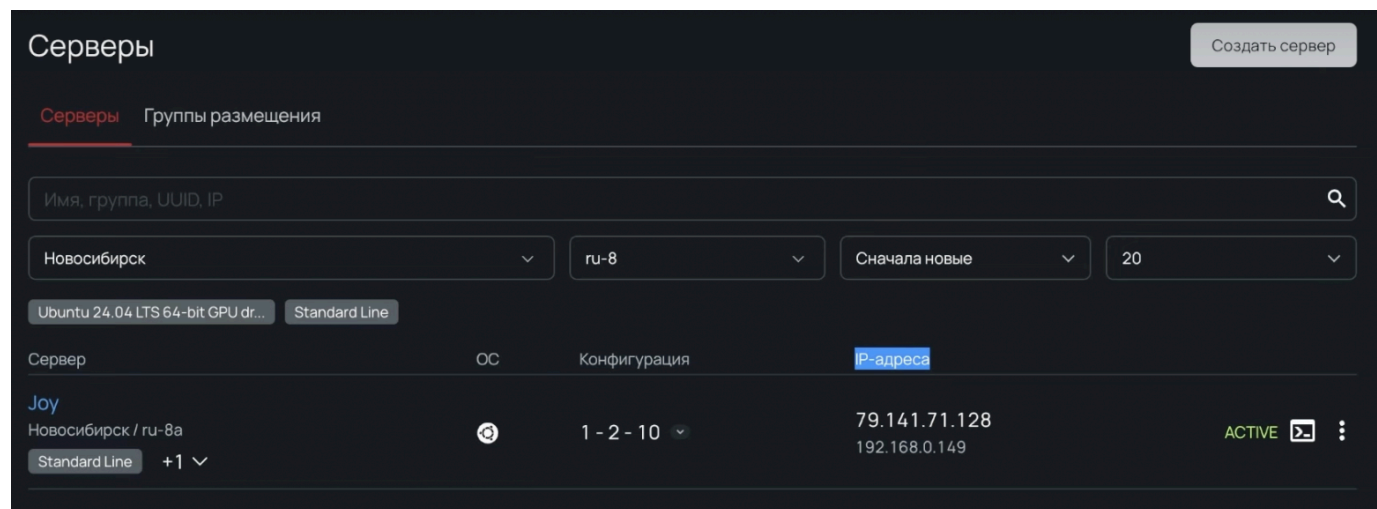
Если вы не нашли подходящую конфигурацию, обратитесь [в службу поддержки](#), и мы подберем решение.

3. Теперь нужно настроить конфигурацию. Регион, пул и название указывайте, исходя из своих предпочтений. Остальные параметры рекомендуем устанавливать следующим образом.

- **1 vCPU и 2 GB RAM** — этого достаточно для стабильной работы NestJS-приложения и обработки голосовых сообщений в фоне. Больше не нужно, так как бот не выполняет ресурсоемких задач постоянно.
- **10 GB SSD** — минимально необходимый объем для хранения ОС, логов, временных аудиофайлов и кэша.
- **Ubuntu 24** — этот дистрибутив легко настраивается и отлично совместим с большинством библиотек.
- **Публичный IP** — нужен, чтобы Telegram мог отправлять запросы на наш сервер. Без него бот не будет получать сообщения.

4. Поскольку в качестве ОС мы выбрали Ubuntu 24, для подключения к серверу обязательно использовать SSH-ключ. О том, как его сгенерировать, можно узнать в инструкции.

5. Нажимаем на кнопку **Создать сервер** и ждем пару минут, пока статус не сменится на ACTIVE.



6. Далее подключаемся к серверу по SSH. Для этого открываем терминал и вводим следующую команду:

```
ssh root@<ip-адрес>
```

Базовая настройка Ubuntu

1. После подключения к серверу, необходимо обновить список доступных пакетов и их версий из репозитория, указанных в конфигурации системы. Это позволяет системе знать о последних версиях программ и обновлениях.

```
apt update && apt upgrade -y
```

2. Далее установим необходимые пакеты и Node.js:

```
apt install -y curl git unzip htop nano nginx
```

```
curl -L <a href="https://raw.githubusercontent.com/tj/n/master/bin/n">https://raw.githu
```

```
chmod +x /usr/local/bin/n
```

3. Ставим Node.js latest version. Если вы не используете Docker, нужно установить такую же версию, как на вашем компьютере. Иначе могут возникнуть конфликты.

```
n lts
```

```
node -v npm -v
```

4. Устанавливаем Bun. Напомним, что это современный быстрый менеджер пакетов, инструмент для разработки и выполнения JavaScript- и TypeScript-проектов.

```
curl -fsSL https://bun.sh/install | bash
export BUN_INSTALL="$HOME/.bun"
export PATH="$BUN_INSTALL/bin:$PATH"
```

5. Также однозначно понадобится установить PM2 — это популярный менеджер процессов для Node.js-приложений. Он помогает обеспечить стабильную работу серверных приложений и упрощает их администрирование.

```
npm install -g pm2
```

На этом этапе все готово и установлено для деплоя.

Деплой бота на готовый сервер

1. Начнем с клонирования нашего GitHub-репозитория. Это довольно легко сделать, если сам репозиторий публичный. Достаточно ввести одну команду:

```
git clone <ссылка на репозиторий>
```

Если репозиторий приватный, понадобится ввести данные от своего аккаунта или предварительно добавить в профиль на GitHub ранее сгенерированный SSH-ключ. Процесс подробно описан в документации.

```
git clone https://github.com/maxZBS/tsst-voice.git
Cloning into 'tsst-voice'...
remote: Enumerating objects: 27, done.
remote: Counting objects: 100% (27/27), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 27 (delta 0), reused 27 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (27/27), 100.93 KiB | 1.08 MiB/s, done.
```

2. Следующим шагом устанавливаем зависимости с помощью Bun:

```
bun install --production
```

3. Создаем в корне проекта файл .env и прописываем в нем полученные ранее TELEGRAM_BOT_TOKEN и OPENAI_API_KEY.

```
nano .env
```

4. Собираем проект с помощью Bun:

```
bun run build
```

5. Проект можно запустить. Но перед этим важно остановить бота на локальной машине, если он работает.

```
pm2 start dist/main.js --name="telegram-bot"
```

```
root@joy ~/tsst-voice git:(main)±1 (0.523s)
pm2 start dist/main.js --name telegram-bot
[PM2] Starting /root/tsst-voice/dist/main.js in fork_mode (1 instance)
[PM2] Done.
```

id	name	mode	Г	status	cpu	memory
0	telegram-bot	fork	0	online	0%	32.2mb

6. Включаем автозапуск бота. Это полезно, например, на случай перезагрузки сервера.

```
pm2 save
```

Готово! Теперь наш бот круглосуточно работает на сервере. В любой момент можно отправить голосовое сообщение и получить расшифровку с таймкодами.

Заключение

В этой инструкции мы разобрали, как создать простого Telegram-бота с использованием API OpenAI и NestJS в качестве стека. А еще — научились деплоить проекты на облачный сервер и настраивать автозапуск на случай перезагрузки сервера.

Материал объемный. Если вам проще изучить тему в формате видео, смотрите выпуск на канале REG Group .

Теги: selectel, linux, telegram, nestjs, grammy, бот

Хабы: Блог компании Selectel, JavaScript, NestJS, Мессенджеры, Программирование

↑ +45 ↓

42



3



Selectel

IT-инфраструктура для бизнеса

Подписаться

[ВКонтакте](#) [Telegram](#) [Сайт](#)



↑ 222 ↓

Карма

136.5

Рейтинг

@Seleditor

Пользователь

Подписаться



 Комментарии 3

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



MaFrance351

17 часов назад

Накопитель цифровой информации. Самый загадочный прибор на базе МК-85



Средний



6 мин



5.2K

Кейс



+82



19



47



Bright_Translate

12 часов назад

Технический взгляд на отключения интернета в Иране



7 мин



7.8K

Перевод



+44



20



24



BabayMazay

16 часов назад

Любительские французские радиолампы 1920-х годов. Часть 2. Стеклодувное, ртутно-капельный вакуумный насос



Средний



9 мин



1.9K

Ретроспектива



+40



11



9



skovalev

16 часов назад

Ускорители AMD Instinct, прорывные Ethernet-коммутаторы и другие новинки июня

 Простой  6 мин  3.3K

Дайджест

 +32

 3

 3



PaaLadin

11 часов назад

Как выйти из IT?.. и пойти слесарем на завод. Моя попытка дауншифтинга [2\4]

 Средний  6 мин  5.5K

Кейс

 +30

 18

 6



aakireeva

18 часов назад

Тесты, баги, два кота. Как устроено тестирование продуктов в Angie Software

 Простой  15 мин  1K

Обзор

 +25

 12

 2



kliment_01

18 часов назад

Windows Home Server: взгляд на домашний сервер Microsoft, который не прижился

 6 мин  4.5K

 +25

 12

 6



erbanovanastasia

13 часов назад

Как установить GUI на сервер Ubuntu 24.04

 Простой  9 мин  3.1K

Тutorial

+23

36

8



habr_career

13 часов назад

Как перейти в IT без увольнения: курсы для горизонтального перехода и опыт сотрудников Хабра



Простой



10 мин



862

+22

9

0



Sravni_Tech

15 часов назад

Пособие по промпт-инжинирингу для программистов



Простой



38 мин



6K

Перевод

+19

138

7

Показать еще

ИНФОРМАЦИЯ

Сайт

slc.tl

Дата регистрации

15 марта 2010

Дата основания

11 сентября 2008

Численность

1 001–5 000 человек

Местоположение

Россия

Представитель

Влад Ефименко

ССЫЛКИ

GPU в облаке от 29 рублей в час

selectel.ru

Серверы для Data Science за 4,12 Р/час
selectel.ru

Серверы для ML-разработки за 4,12 Р/час
selectel.ru

50 ГБ хранилища от 80 рублей в месяц
selectel.ru

FAQ
slc.tl

Реферальная программа
slc.tl

Telegram-канал о технологиях
t.me

Telegram-канал про карьеру в IT
t.me

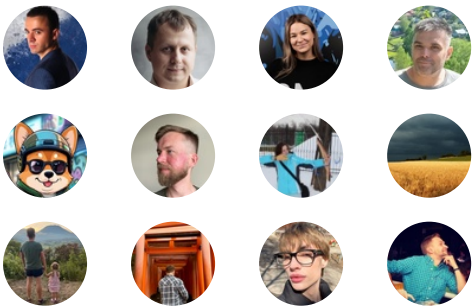
Вакансии
slc.tl

Академия Selectel
slc.tl

В КОНТАКТЕ

 **Selectel**

102 379 подписчиков



Подписаться на новости

ВИДЖЕТ

Домены в Selectel

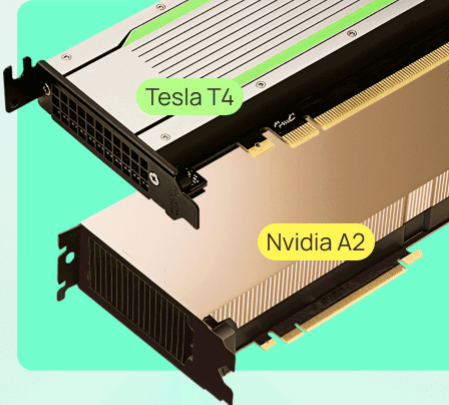


Регистрация,
продление
и перенос доменов
в подарок!

Реклама. erid: 2Vtzqwh859R. АО Селектел

ВИДЖЕТ

Selectel



Tesla T4

Nvidia A2

GPU за 1 рубль

При заказе сервера произвольной конфигурации в Selectel в рамках Акции при выборе GPU Tesla T4 либо GPU NVIDIA A2. Предложение действует до 31.07.2025. Количество доступных для заказа в рамках Акции GPU-карт ограничено.

[Подробнее](#)

Реклама. erid: 2Vtzqvo8o6X. АО Селектел

ВИДЖЕТ



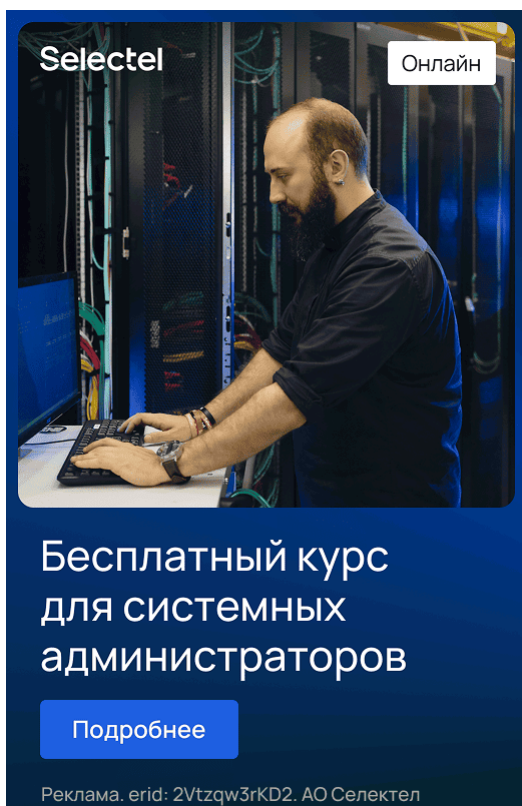
Облачный
сервер Selectel

с криперами и порталом в Незер

Исследовать

Реклама. 2VtzquqtAPR. АО Селектел

ВИДЖЕТ



Selectel

Онлайн

Бесплатный курс
для системных
администраторов

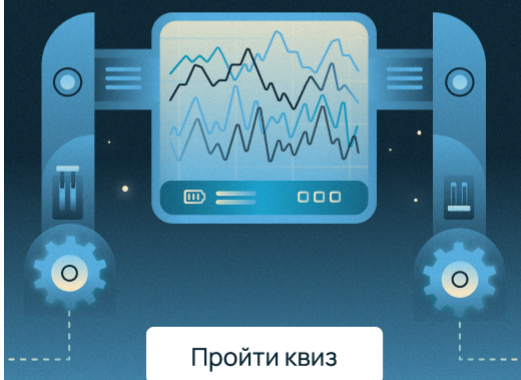
Подробнее

Реклама. erid: 2Vtzqw3rKD2. АО Селектел

ВИДЖЕТ

Готовы проверить знания в ML?

Пройдите тест и получите
полезные материалы



Реклама. erid: 2Vtzqvid8xF. АО Селектел

ВИДЖЕТ

IaaS

PaaS

Хранение

Бесплатное S3-хранилище на 30 дней

Для всех, кто ранее
не использовал услугу
в Selectel



Оставить заявку

Реклама. erid: 2VtzqvVKLsV. АО Селектел

БЛОГ НА ХАБРЕ

13 часов назад

Как установить GUI на сервер Ubuntu 24.04

 3.1K  8

16 часов назад

Ускорители AMD Instinct, прорывные Ethernet-коммутаторы и другие новинки июня

 3.3K  3

17 июл в 15:00

Ruff: мой опыт выселения legacy-линтеров и повышения производительности кода

 2.1K  3

17 июл в 11:01

Автоматизируем сканирование IT-инфраструктуры: скрипт v2.0 для Сканер-ВС 6

 1.9K  2

Ваш аккаунт	Разделы	Информация	Услуги
Профиль	Статьи	Устройство сайта	Корпоративный блог
Трекер	Новости	Для авторов	Медийная реклама
Диалоги	Хабы	Для компаний	Нативные проекты
Настройки	Компании	Документы	Образовательные
ППА	Авторы	Соглашение	программы
	Песочница	Конфиденциальность	Стартапам



Настройка языка

Техническая поддержка